
Table of Contents

.....	1
-------	---

```
clear all
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

The following line of code loads a previously generated set of fictitious data.

```
addpath('mcmcstat')
addpath('kde')

load spring_model_data %this is a set of dynamic data

data.ydata = y_data2; %stress in Pa
data.xdata = t; %strain in mm/mm
data.xdata = data.xdata(:);
data.ydata = data.ydata(:);

%define initial guess for parameters
y0 = 1.4784; %initial conditions
k = 313^2; %spring constant
C = 27.916; %damping term
m = 1;

% theta = [y0
%          k
%          C
%          m
%          ];
theta = [y0
         k
         C
         ];

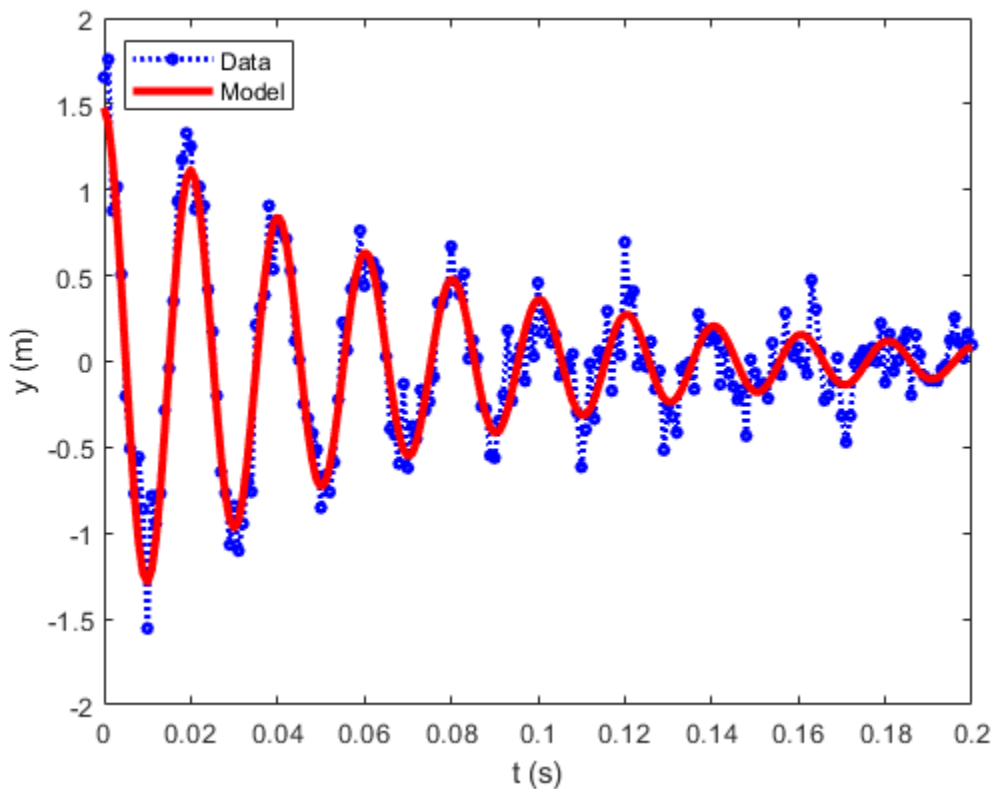
%model parameter range
% params = {
%   {'y_0', theta(1), 0, inf}
%   {'k', theta(2), 0, 300^2*100}
%   {'C', theta(3), 0, inf}
%   {'m', theta(4), 0, 100}
%   };
params = {
    {'y_0', theta(1), 0, inf}
    {'k', theta(2), 0, 300^2*100}
    {'C', theta(3), 0, inf}
    };

Call function that computes the error between the model and data

ssfun = @SS_func;
model.ssfun=ssfun;
```

The next lines of code check the initial parameter guesses prior to running Bayesian parameter calibration to see if the values given reasonable results with respect to the data.

```
[y_model] = mass_spring_model_Bayesian(theta,data.xdata);  
% [Chi] = mass_spring_model_sensitivity(theta,data.xdata);  
  
figure(1)  
plot(data.xdata,y_data2,'bo:','MarkerSize',3,'Linewidth',2)  
hold on  
plot(data.xdata,y_model,'r-','Linewidth',3)  
hold off  
xlabel('t (s)')  
ylabel('y (m)')  
legend('Data','Model','Location','NorthWest')  
saveas(gcf, "ModelAndData_reduced.jpeg");  
  
% figure(10)  
% plot(data.xdata,Chi,'bo:','MarkerSize',3,'Linewidth',2)  
% xlabel('t (s)')  
% ylabel('dy/dC')
```



The Bayesian analysis is calculated here.

```
model.sigma2 = 1e-4;           %initial guess on variance  
model.S20 = model.sigma2;      %prior for sigma2  
model.N0 = 1;                  %prior accuracy for S20  
options.updatesigma = 1;       %update variance as part of the inference
```

```

options.method = 'dram'; %this applies the DRAM algorithm
model.N = length(data.xdata); %number of data points

options.nsimu = 50000; %number of iterations in the Metropolis method
[results, chain, s2chain]= mcmcrun(model,data,params,options);
chainstats(chain,results) %print chain statistics

```

Sampling these parameters:

```

name    start [min,max] N(mu,s^2)
y_0: 1.4784 [0,Inf] N(0,Inf)
k: 97969 [0,9e+06] N(0,Inf)
C: 27.916 [0,Inf] N(0,Inf)
MCMC statistics, nsimu = 50000

```

	mean	std	MC_err	tau	geweke
y_0	1.5248	0.058133	0.00061862	9.1494	0.99873
k	98336	372.83	7.2856	9.1507	0.9995
C	29.549	1.6747	0.017673	8.9852	0.99602

Plot the statistical results. Note that Figure 4 containing pair correlations will not plot except for the nonlinear case where there is more than one parameter.

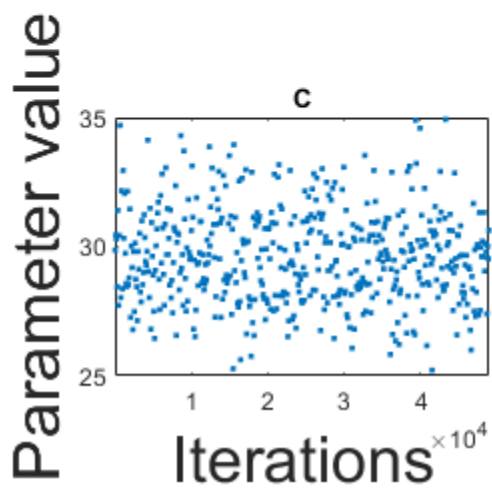
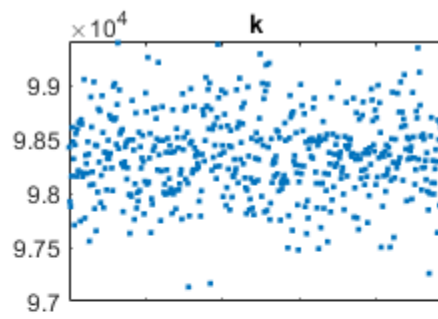
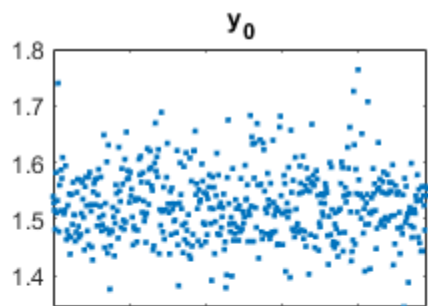
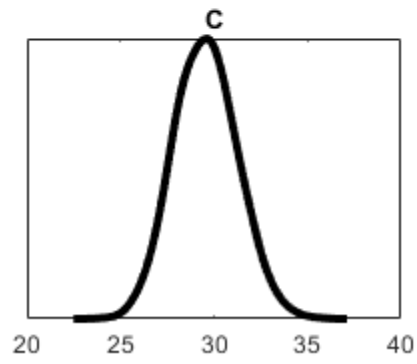
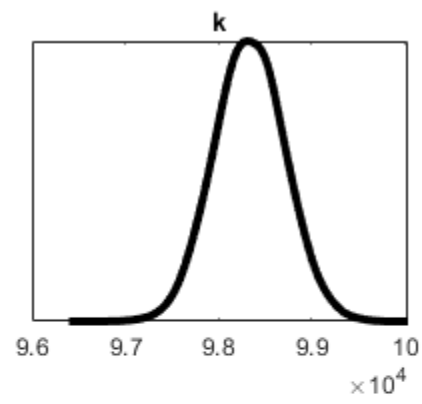
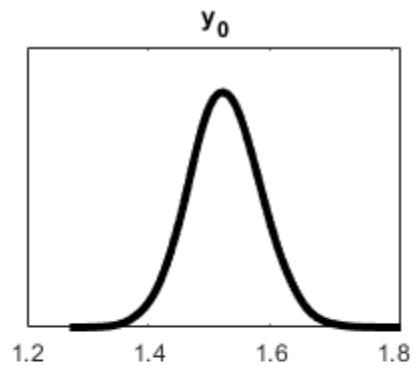
```

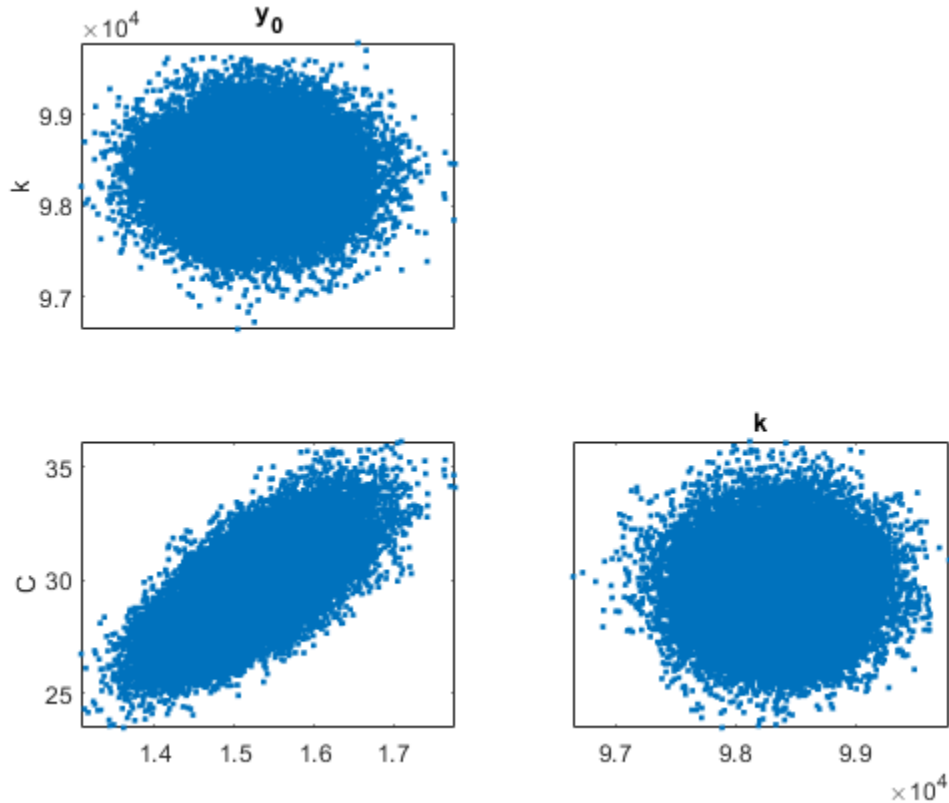
figure(2)
mcmcplot(chain(1000:end,:),[],results,'denspanel',2);
saveas(gcf, "Density_reduced.jpeg");

figure(3); clf
mcmcplot(chain(1000:end,:),[],results.names,'chainpanel')
xlabel('Iterations','FontSize',24)
ylabel('Parameter value','FontSize',24)
saveas(gcf, "Chain_reduced.jpeg");

figure(4)
mcmcplot(chain,[],results,'pairs');
saveas(gcf, "Pairs_reduced.jpeg");

```





Compute the credible and prediction intervals

```

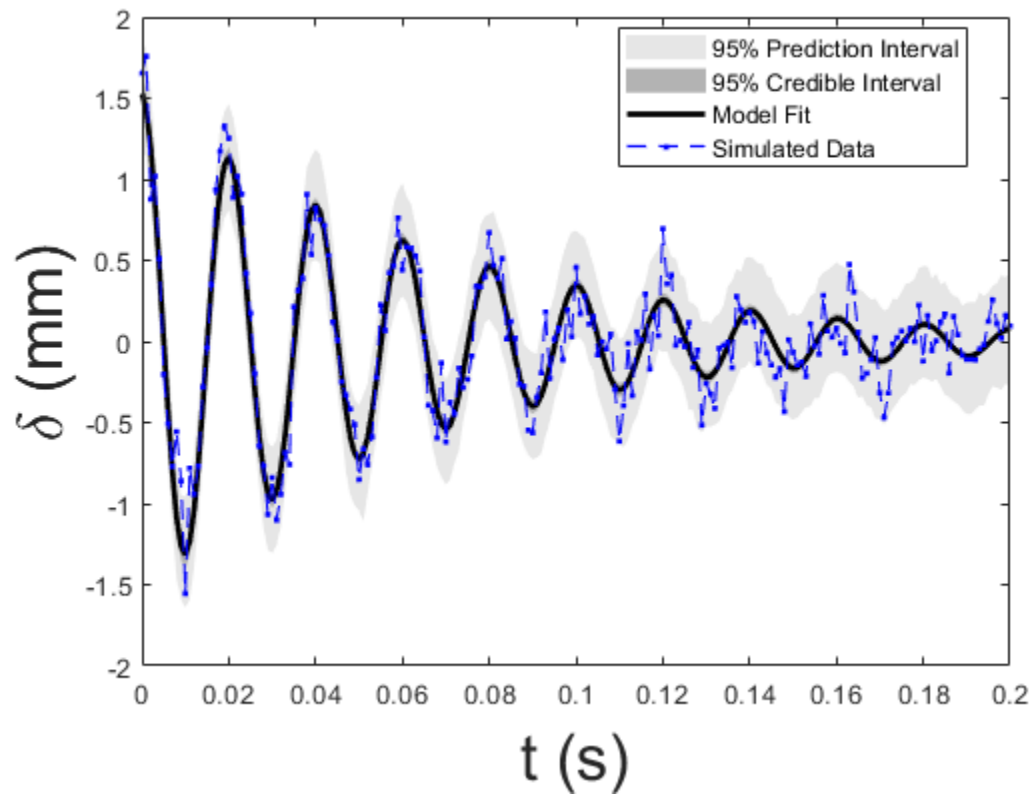
modelfun1 = @(d,th)mass_spring_model_Bayesian(th,d); % NOTE: the order in
which d and th appear is important

nsample = 500; %number of sample iterations of the model used to construct
the interval bounds
    %the default interval bounds are 95% prediction/credible
    %bounds
out = mcmcpred(results,chain,s2chain,data.xdata,modelfun1,nsample);
figure(5)
modelout = mcmcpredplot(out);
hold on
plot(data.xdata,data.ydata,'b.--','linewidth',1)
hold off
xlabel('t (s)','FontSize',24);
ylabel('\delta (mm)','FontSize',24);
legend('95% Prediction Interval','95% Credible Interval','Model
Fit','Simulated Data','Location','Best')
saveas(gcf, "Error_reduced.jpeg");

% %Estimate distribution of C using normal distribution
% sig02 = mean(s2chain)/(Chi'*Chi);
% C0 = 29.342;
% C_dist = C0 + sig02*randn(500,1);
% Range = max(C_dist) - min(C_dist);
% MinC = min(C_dist)-Range/10

```

```
% MaxC = max(C_dist)+Range/10
% [bandwidth,C_density,xmesh,cdf] = kde(C_dist,2^14,MinC,MaxC);
%
% figure(20)
% plot(xmesh,C_density,'r')
```



Published with MATLAB® R2024b